

5. PROJECT DEVELOPMENT PHASE

5.1 Development Environment and Project Structure

Development begins by installing the selected programming language and required libraries, creating an isolated environment and recording dependencies. Source code should be organized into modules for interface pages, database access, validation, statistical calculations and report generation. This structure reduces duplication and makes individual components easier to test.

A practical folder structure may include the application entry point, page components, database utilities, calculation utilities, assets, tests and documentation. Database creation should use a repeatable initialization script. Sample records should be loaded separately so that the demonstration environment can be recreated without manually entering all data each time.

Configuration values should be separated from business logic. Credentials and secrets should not be committed to source control. Logging should help diagnose errors without recording sensitive values. Naming conventions and formatting practices should be agreed early so that different modules remain consistent.

Each feature can be developed as a small task, tested locally and reviewed before integration. The setup guide should be updated as dependencies or configuration change. A clean installation test near the end confirms that another user can recreate the environment using the documented steps.

5.2 Implementing Player, Team and Match Management

Player management supports creating profiles, viewing details, editing permitted attributes and searching by name or identifier. Each player should have a stable ID. Duplicate checks can help prevent repeated profiles caused by spelling differences. Profile fields should be limited to information relevant to the application, such as playing role and batting or bowling style.

Team management supports team creation and player association. If the application tracks seasons or transfers, membership should be time-aware so that historical match records remain accurate. A player's current team should not automatically overwrite the team represented in an earlier match.

Match management records date, format, venue and participating teams. The system should verify that teams are distinct and that innings belong to the selected match. Match status may be used to distinguish scheduled, in-progress and completed records if those states are needed. Editing completed matches should be controlled because changes affect reports.

Forms should confirm successful operations and preserve valid entered values when validation fails. Search and filtering should behave consistently across pages. The initial version should prioritize reliable forms and tables before adding advanced bulk import or complex workflows. Every saved record should be retrievable and visible in related views.

5.3 Implementing Performance Entry and Calculations

Performance entry should follow the chosen scorecard structure. Batting records may include runs, balls faced, fours, sixes and dismissal status. Bowling records may include legal balls, maidens, runs conceded and wickets. Fielding records can include catches, stumpings and run-outs when supported by the data source. The interface should distinguish entered values from calculated results.

Calculation functions should be independent of the interface. Batting strike rate is runs divided by balls faced multiplied by 100. Batting average is runs divided by dismissals. If the denominator is zero, the application should show an agreed unavailable value such as N/A. Bowling economy should use legal balls, not decimal interpretation of overs notation.

For example, 4.2 overs means four completed overs and two additional legal balls, or 26 legal balls. Economy can be calculated as runs conceded multiplied by six and divided by legal balls. The application should define how wides, no-balls and other extras affect the stored values according to the selected scorecard convention.

Tests should use verified examples and include edge cases. If a user corrects an input, dependent summaries should update. Derived statistics should be calculated from canonical source values rather than manually entered wherever practical. Incomplete records should be identified so users do not mistake partial information for a finalized performance.

5.4 Dashboard, Reports and Export

The dashboard provides a concise overview of available records, such as recent matches, player count and selected performance summaries. It should state the period or filters used and avoid implying that a small sample proves long-term ability. Player reports can show match-by-match figures and aggregates for a selected date range or season.

Team reports may summarize match results and player contributions if the required data is available. Filters should be clearly labeled and applied consistently. Reports should identify the player or team, date range, match count and metric definitions. If no records match the selected filters, the application should display a helpful empty state rather than a misleading zero-valued report.

Charts should use suitable labels and units. A runs-per-match chart can show match date or sequence against runs. Bowling economy, wickets and runs conceded represent different measures and should be presented separately or with a carefully explained design. Every chart should have a title and should reflect the same filters as the surrounding report.

CSV export supports further analysis in spreadsheet software. Column names should be descriptive, dates consistent and missing values represented clearly. If PDF export is implemented, include a report title, generation date and filter scope. Export should be tested with empty results, special characters in names and larger datasets to ensure that the file matches the displayed report.